

Access Logging — Tomcat vs Netty

WHAT IS ACCESS LOGGING

Access Log INFRASTRUCTURE LAYER

Generated automatically by the **server layer** — not your application code. Records every HTTP request regardless of business logic. One line per completed request.

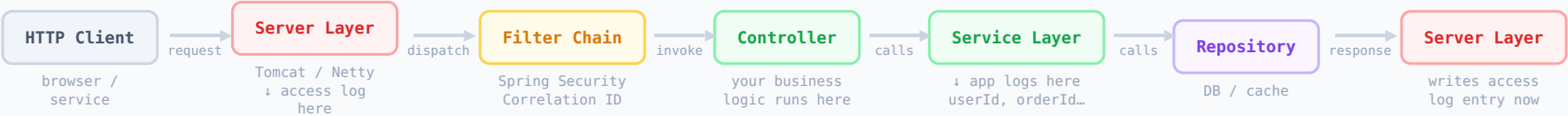
```
192.168.1.1 - - [05/Jun/2026:10:23:45 +0000]
"GET /api/orders/123 HTTP/1.1" 200 1423 45ms
"POST /api/payment HTTP/1.1" 500 89 230ms
```

Application Log APPLICATION LAYER

Written by **your code** via SLF4J/Logback. Captures business context, decisions, errors — things only your application knows about.

```
INFO OrderService - Order 123 fetched for user u_789
ERROR PaymentService - Stripe timeout after 3 retries
DEBUG InventoryService - Stock check: 3 units remaining
```

REQUEST FLOW — WHERE EACH LOG IS WRITTEN



Access log is written at the **server layer envelope** — it sees IP, method, path, status, bytes, duration. It has zero visibility into `userId` or `orderId`. **Application log** is written inside your code — it knows business context but doesn't automatically know the HTTP method or client IP unless you add it. Both are needed. Neither replaces the other.

KEY DIFFERENCES AT A GLANCE

ASPECT	ACCESS LOG	APPLICATION LOG
Who writes it	Server runtime (Tomcat / Netty) — automatic	Your code via <code>log.info()</code> / <code>log.error()</code>

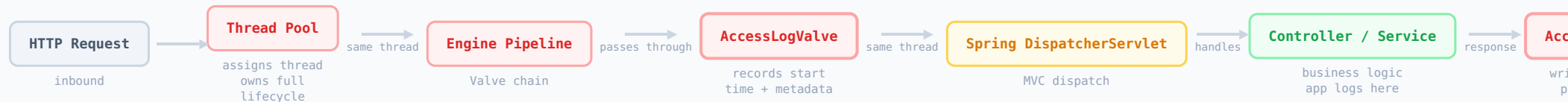
Triggered by	Every HTTP request, unconditionally	Only when your code executes a log statement
Contains	IP, method, path, status code, bytes, latency	Business context: userId, orderId, error details
Format	Fixed (Combined Log Format or pattern)	Configurable via Logback / Log4j2 pattern
Volume	Exactly one line per request	Zero to many lines per request
Default destination	File (must configure for console/stdout)	Console by default in Spring Boot

Tomcat Access Logging

SPRING MVC · THREAD-PER-REQUEST · ACCESSLOGVALVE

02 / 05 · Tomcat

ARCHITECTURE — THREAD-PER-REQUEST MODEL



Key property: Because Tomcat assigns one thread per request, the AccessLogValve runs on the **same thread** as your application code – from request arrival to response completion. This makes MD available to both access and application logs.

SPRING BOOT CONFIGURATION — SERVER.TOMCAT.ACCESSLOG.*

APPLICATION.PROPERTIES

```
# — Enable access log (disabled by default) — server.tomcat.accesslog.enabled=true
# — Console output (stdout) — K8s recommended — # Write to /dev/stdout via file
# trick server.tomcat.accesslog.directory=/dev server.tomcat.accesslog.prefix=stdout
server.tomcat.accesslog.suffix= server.tomcat.accesslog.file-date-format=
server.tomcat.accesslog.buffered=false # — Log pattern — # %h=remote host
%l=identity %u=user # %t=timestamp %r=request %s=status # %b=bytes %D=duration ms
server.tomcat.accesslog.pattern=%h %l %u %t "%r" %s %b %D # — Rotation (when writing
to file) — server.tomcat.accesslog.rotate=true server.tomcat.accesslog.rename-on-
rotate=false server.tomcat.accesslog.max-days=30 # — Tomcat internal debug logs —
logging.level.org.apache.tomcat=INFO logging.level.org.apache.catalina=INFO
```

JAVA CONFIG — ROUTE THROUGH SLF4J (PREFERRED)

```
@Configuration public class TomcatAccessLogConfig { private static final Logger log =
LoggerFactory.getLogger("ACCESS"); @Bean public
WebServerFactoryCustomizer<TomcatServletWebServerFactory> accessLogCustomizer() {
return factory -> factory.addEngineValves( new AbstractAccessLogValve() { @Override
protected void log(CharArrayWriter message) { // Flows through Logback — same
appender // as application logs. No /dev/stdout trick.
log.info(message.toString()); } }); } }
```

APPLICATION.PROPERTIES — LOGBACK LOGGER FOR ACCESS

```
# Suppress default Tomcat file valve if using Java config
server.tomcat.accesslog.enabled=false # Set level for the ACCESS logger used above
logging.level.ACCESS=INFO
```

Netty Access Logging

SPRING WEBFLUX · EVENT-LOOP · WEBFILTER / REACTOR NETTY

03 / 05 · Netty

ARCHITECTURE – EVENT-LOOP NON-BLOCKING MODEL



Key difference from Tomcat: There is no single thread owning the request. The event loop processes many requests concurrently. The `doFinally()` operator in the WebFilter fires after the reactive chain completes – this is the Netty equivalent of Tomcat's post-request hook. MDC does not work reliably here; use Reactor `contextWrite()` for correlation IDs.

OPTION A – REACTOR NETTY BUILT-IN

JVM SYSTEM PROPERTY (NOT A SPRING BOOT PROPERTY)

```
# application.properties – does NOT work for this # Must be set as a JVM system property
# Maven / Gradle run plugin -
Dreactor.netty.http.server.accessLogEnabled=true # Or in Spring Boot Maven Plugin
<configuration> <jvmArguments> -Dreactor.netty.http.server.accessLogEnabled=true </jvmArguments> </configuration>
# Or programmatically in main() before context starts
System.setProperty( "reactor.netty.http.server.accessLogEnabled", "true" );
```

LOGBACK-SPRING.XML – ROUTE TO CONSOLE

```
<logger name="reactor.netty.http.server.AccessLog" level="INFO" additivity="false">
  <appender-ref ref="CONSOLE"/> </logger>
```

Built-in option writes Combined Log Format. Limited format control. Logger name is `reactor.netty.http.server.AccessLog`.

OPTION B – CUSTOM WEBFILTER (RECOMMENDED)

ACCESSLOGFILTER.JAVA

```
@Component @Order(Ordered.HIGHEST_PRECEDENCE) public class AccessLogFilter implements WebFilter {
  private static final Logger log = LoggerFactory.getLogger("ACCESS");
  @Override public Mono<Void> filter( ServerWebExchange exchange, WebFilterChain chain) {
    long start = System.currentTimeMillis(); String traceId = UUID.randomUUID().toString();
    return chain.filter(exchange) .doFinally(signal -> {
      long ms = System.currentTimeMillis() - start; ServerHttpRequest req = exchange.getRequest();
      ServerHttpResponse res = exchange.getResponse(); log.info("{} {} {} {}ms [{}]", req.getMethod(), req.getURI().getPath(), res.getStatusCode(), ms, traceId);
    }) .contextWrite(Context.of( "traceId", traceId)); } }
```

Full Configuration Reference

ALL PROPERTIES · SPRING BOOT · LOGBACK · JVM FLAGS

04 / 05 · Config Reference

TOMCAT — APPLICATION.PROPERTIES

```
# — Core ————— # Prefix:
server.tomcat.accesslog.*
server.tomcat.accesslog.enabled=true
server.tomcat.accesslog.buffered=false # — Console output
(stdout) ————— server.tomcat.accesslog.directory=/
dev server.tomcat.accesslog.prefix=stdout
server.tomcat.accesslog.suffix=
server.tomcat.accesslog.file-date-format= # — File output
————— #
server.tomcat.accesslog.directory=logs #
server.tomcat.accesslog.prefix=access_log #
server.tomcat.accesslog.suffix=.log #
server.tomcat.accesslog.file-date-format=.yyyy-MM-dd # —
Pattern tokens ————— # %h remote host IP
# %l identity (usually -) # %u remote user # %t timestamp #
# %r first line of request # %s HTTP status code # %b bytes
sent # %D time taken (ms) # %T time taken (seconds) # %{X-
Trace-Id}i request header # %{X-Trace-Id}o response header
server.tomcat.accesslog.pattern=%h %l %u %t "%r" %s %b %D #
— Rotation —————
server.tomcat.accesslog.rotate=true
server.tomcat.accesslog.max-days=30
server.tomcat.accesslog.rename-on-rotate=false # —
Conditional logging ————— # Only log if header
present server.tomcat.accesslog.condition-if=X-Log-Request
# Skip if header present server.tomcat.accesslog.condition-
unless=X-Health-Check # — Tomcat internal logs
————— logging.level.org.apache.tomcat=INFO
logging.level.org.apache.catalina=INFO
logging.level.org.apache.coyote=WARN
```

NETTY — JVM PROPERTY + APPLICATION.PROPERTIES

```
# — IMPORTANT ————— #
reactor.netty.http.server.accessLogEnabled # is NOT a
Spring Boot property. # It must be set as a JVM system
property. # application.properties CANNOT enable it. # —
JVM flag (command line) ————— -
Dreactor.netty.http.server.accessLogEnabled=true # —
Spring Boot Maven Plugin ————— # In pom.xml under
spring-boot-maven-plugin # <configuration> # <jvmArguments>
# -Dreactor.netty.http.server.accessLogEnabled=true # </
jvmArguments> # </configuration> # — Programmatic (before
app context) — # In main() before
SpringApplication.run(): # System.setProperty( #
"reactor.netty.http.server.accessLogEnabled", # "true"); #
— application.properties (Spring props) — # WebFlux server
settings use server.* prefix spring.webflux.base-path=/api
server.port=8080 # — Netty internal debug
————— logging.level.reactor.netty=INFO
logging.level.reactor.netty.http.server=INFO # — Route
Netty access log to console — # In logback-spring.xml
(see next column) # Logger:
reactor.netty.http.server.AccessLog # — WebFilter approach
(no JVM flag needed) # Implement WebFilter with doFinally()
# Add @Order(Ordered.HIGHEST_PRECEDENCE) # Use
contextWrite() for correlation IDs
```

LOGBACK-SPRING.XML — BOTH SERVERS

```
<?xml version="1.0"?> <configuration> <!-- Console appender
--> <appender name="CONSOLE"
class="ch.qos.logback.core.ConsoleAppender"> <encoder>
<pattern>%d{HH:mm:ss} %-5p [%t] %c{1} - %msg%n</pattern> </
encoder> </appender> <!-- Separate ACCESS console appender
--> <appender name="ACCESS_CONSOLE"
class="ch.qos.logback.core.ConsoleAppender"> <encoder> <!--
Plain format for access logs --> <pattern>%msg%n</pattern>
</encoder> </appender> <!-- Netty built-in access log
logger --> <logger
name="reactor.netty.http.server.AccessLog" level="INFO"
additivity="false"> <appender-ref ref="ACCESS_CONSOLE"/> </
logger> <!-- Custom ACCESS logger (Tomcat valve --> <!-- or
Netty WebFilter approach) --> <logger name="ACCESS"
level="INFO" additivity="false"> <appender-ref
ref="ACCESS_CONSOLE"/> </logger> <!-- Root logger --> <root
level="INFO"> <appender-ref ref="CONSOLE"/> </root> </
configuration>
```

Comparison + Correlation ID Strategy

SIDE-BY-SIDE COMPARISON

ASPECT	TOMCAT (MVC)	NETTY (WEBFLUX)
Thread model	One thread per request — thread owns full lifecycle	Event loop — thread is shared across many requests
Built-in mechanism	AccessLogValve in Tomcat Engine pipeline	No built-in valve; use WebFilter or reactor.netty JVM flag
Spring Boot property prefix	<code>server.tomcat.accesslog.*</code>	No Spring property — JVM flag only for built-in
Enable via application.properties	Yes — <code>server.tomcat.accesslog.enabled=true</code>	No — must use JVM -D flag or <code>System.setProperty()</code>
Console (stdout) output	/dev/stdout trick OR custom AbstractAccessLogValve → SLF4J	Built-in logs to reactor.netty.http.server.AccessLog logger; WebFilter logs anywhere
Correlation ID injection	MDC.put() — works reliably on same thread	contextWrite(Context.of("tracelid", id)) — MDC unreliable
Log timing accuracy	Post-request on same thread — always complete	doFinally() after reactive chain — complete including async ops
Format customisation	Pattern tokens (%h %r %s %D) or full custom valve	Full control via WebFilter; built-in is Combined Log Format only
K8s / container recommendation	Use /dev/stdout trick or AbstractAccessLogValve → SLF4J	Use WebFilter → SLF4J; no file I/O in container
Dependency	spring-boot-starter-web (includes Tomcat)	spring-boot-starter-webflux (includes Reactor Netty)

CORRELATION ID — JOINING ACCESS + APPLICATION LOGS

Tomcat — MDC on Same Thread

SPRING MVC

```
@Component @Order(Ordered.HIGHEST_PRECEDENCE) public class CorrelationFilter
implements Filter { @Override public void doFilter(ServletRequest req,
ServletResponse res, FilterChain chain) { String id = UUID.randomUUID().toString();
MDC.put("traceId", id); ((HttpServletResponse) res) .setHeader("X-Trace-Id", id);
try { chain.doFilter(req, res); } finally { MDC.clear(); // always clean
up } } } // In application.properties pattern: // Include %[X-Trace-Id]o in
accesslog pattern // Include %X{traceId} in logback pattern
```

Netty — Reactor Context

SPRING WEBFLUX

```
@Component @Order(Ordered.HIGHEST_PRECEDENCE) public class CorrelationFilter
implements WebFilter { @Override public Mono<Void> filter( ServerWebExchange ex,
WebFilterChain chain) { String id = ex.getRequest().getHeaders() .getFirst("X-
Trace-Id"); if (id == null) id = UUID.randomUUID().toString();
ex.getResponse().getHeaders() .add("X-Trace-Id", id); final String traceId = id;
return chain.filter(ex) .contextWrite(Context.of( "traceId", traceId)); //
Retrieve: Mono.deferContextual(ctx -> // ctx.get("traceId")) } }
```